

Databases

Contents

1	The Entity-Relationship (ER) model	4
1.1	Entities	4
1.2	Relationships	4
1.2.1	Binary relationships	4
1.2.2	n-ary relationships	5
1.2.3	Unary relationships	5
1.2.4	Role names	6
1.3	Attributes	6
1.3.1	Keys	7
1.3.2	Attributes on entity diagrams	7
1.3.3	Attributes on relationships	7
1.4	Multiplicity	8
1.4.1	Multiplicity of n-ary relationships	9
1.5	Problems with the ER model	9
1.6	The Enhanced Entity-Relationship (EER) model	9
1.6.1	Superclasses and subclasses	9
1.6.2	Advantages	10
1.6.3	Specialisation	10
1.6.4	Generalisation	10
1.6.5	Constraints	11
1.6.6	EER diagrams	11
2	Semi-structured data - XML	12
2.1	Types of data organisation	12
2.1.1	Structured data	12
2.1.2	Semi-structured data	12
2.1.3	Unstructured data	12
2.2	eXtensible Markup Language (XML)	13
2.3	Tags and terminology	13
2.4	XML data as a graph	13
2.5	Advantages of XML	14
2.6	XML attributes	14
2.6.1	Attributes vs elements	15
2.6.2	ID and IDREF	15
2.7	More XML syntax	15
2.7.1	Comments	15

2.7.2	CDATA	16
2.7.3	Entity reference	16
2.8	XML validation	16
2.8.1	Document Type Definition (DTD)	16
2.8.2	XML schema	18
2.8.3	Well-formed XML	18
2.8.4	Type-valid XML	18
2.8.5	Schema-valid XML	18
3	XPath	18
3.1	XPath queries	19
3.1.1	Paths	19
3.1.2	Descendant search	20
3.1.3	Wildcards	20
3.1.4	Attribute nodes	21
3.1.5	Predicates	21
3.1.6	Multiple predicates	22
3.1.7	Axes	22
3.1.8	Functions	23
3.1.9	Positional predicates	24
4	XQuery	24
4.1	Basic query form	24
4.1.1	FOR-WHERE-RETURN	25
4.1.2	FOR vs LET	25
4.1.3	Collections	26
4.1.4	WHERE	26
4.1.5	Joins	27
4.1.6	Ordering	27
4.1.7	IF-THEN-ELSE	27
4.1.8	Existential qualifiers: SOME	28
4.1.9	Universal qualifiers: EVERY	28
5	Transactions	28
5.1	Database Management Systems	28
5.2	Transactions	29
5.2.1	Properties of transactions	29
5.3	Concurrency control	30
5.3.1	Lost update problem	30
5.3.2	Uncommitted dependency problem	31
5.3.3	Inconsistent analysis problem	32
5.4	Schedules	33
5.4.1	Serialisable schedules	34
5.5	Concurrency control approaches	35
5.6	Locking	35
5.6.1	Rules for locks	35
5.6.2	Two-phase locking	36
5.6.3	Cascading rollback problem	36

5.6.4	Rigorous two-phase locking	37
5.7	Deadlocks	37
5.7.1	Handling deadlocks	37
5.8	Recoverable schedules	38
5.9	Cascadeless schedules	38
5.10	Schedules and consistency	38
5.11	Timestamping	39
5.11.1	Read rule	39
5.11.2	Write rule	39
5.11.3	Correctness of timestamping	40
5.11.4	Recoverability in timestamping	40
5.11.5	Thomas' write rule	40
5.12	Multiversion timestamps	41
5.12.1	Data structure	41
5.12.2	Read operation	41
5.12.3	Writing in multiversion timestamps	42
5.12.4	Version deletion	42
5.13	Optimistic concurrency control	42
5.13.1	Read phase	42
5.13.2	Validation phase	43
5.13.3	Write phase	43
5.14	Validation	43
5.14.1	Validation conditions	43
5.14.2	Properties of optimistic validation	44
5.15	Advanced transaction models	44
5.15.1	Problems with long transactions	45
5.15.2	Nested transaction model	45
5.15.3	Sagas	47
5.15.4	Dynamic restructuring	47
5.15.5	Workflow models	48
6	Distributed databases	49
6.1	Distributed transactions	49
6.2	Distributed serialisability	50
6.2.1	Testing serialisability	50
6.2.2	Ensuring serialisability	50
6.2.3	Centralised 2PL	51
6.2.4	Primary copy 2PL	51
6.2.5	Distributed 2PL	52
6.2.6	Majority 2PL	53
6.3	Distributed deadlocks	53
6.3.1	Centralised deadlock detection	53
6.3.2	Hierarchical deadlock detection	54
6.3.3	Distributed deadlock detection	54
6.4	Network partitioning	55
6.4.1	Two-phase commit	56

1 The Entity-Relationship (ER) model

The Entity-Relationship (ER) model is a top-down approach to database design that provides a graphical description of a database.

It shows:

- **entities:** the important data objects
- **attributes:** the important properties of the entities
- **relationships:** the associations between entities
- **constraints:** rules that apply to entities, attributes, and relationships

1.1 Entities

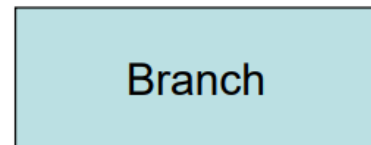
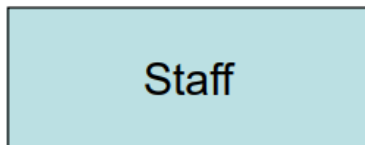
An **entity type** is a group (or class) of objects with the same properties that are identified as having independent existence.

- Example: **Client**

An **entity** occurrence is a uniquely identifiable instance of an entity type

- Example: a specific client called **James Smith**

An entity type is represented in an ER diagram by a rectangle labelled with the entity type name.



1.2 Relationships

A **relationship** is a meaningful association between two (or more) entity types.

The **degree** of a relationship is the number of entity types participating in the relationship.

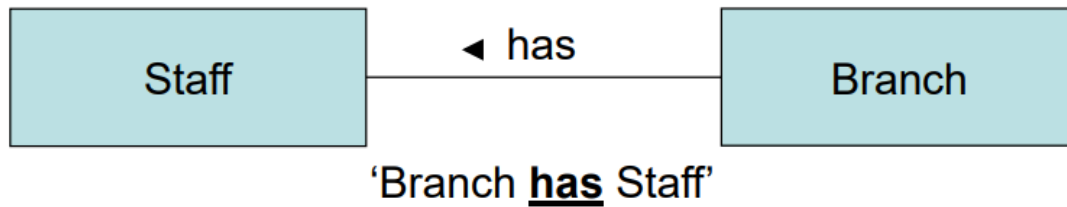
A relationship with degree:

- 1 is called a unary relationship
- 2 is called a binary relationship
- 3 is called a ternary relationship
- n is called an n-ary relationship

1.2.1 Binary relationships

A binary relationship is represented by a line connecting the participating entities, labelled with the name of the relationship.

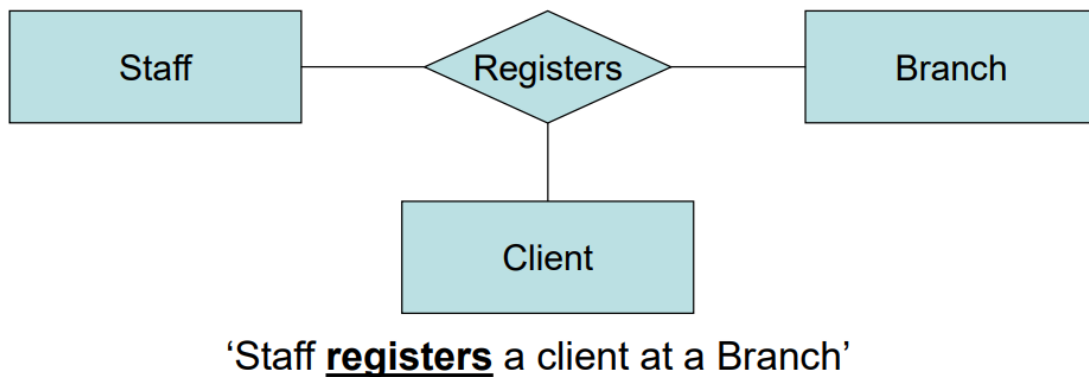
The relationship label may also indicate a reading direction.



1.2.2 n-ary relationships

To represent an n-ary relationship where $n \geq 3$, a diamond symbol labelled with the relationship name is used to connect the participating entities.

An n-ary relationship does not necessarily have a direction.

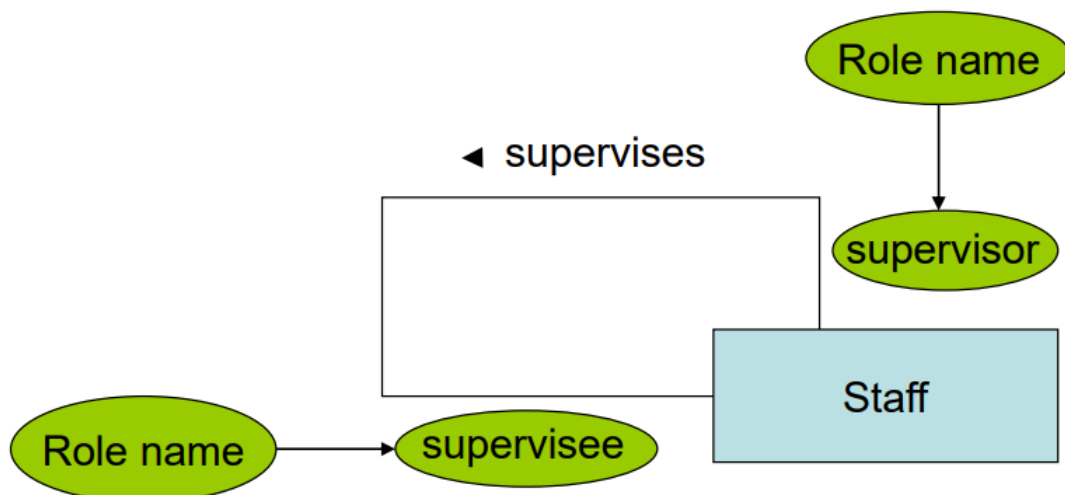


1.2.3 Unary relationships

For unary (or recursive) relationships, an entity is related to itself.

This is represented by a relationship line looping from the entity back to itself.

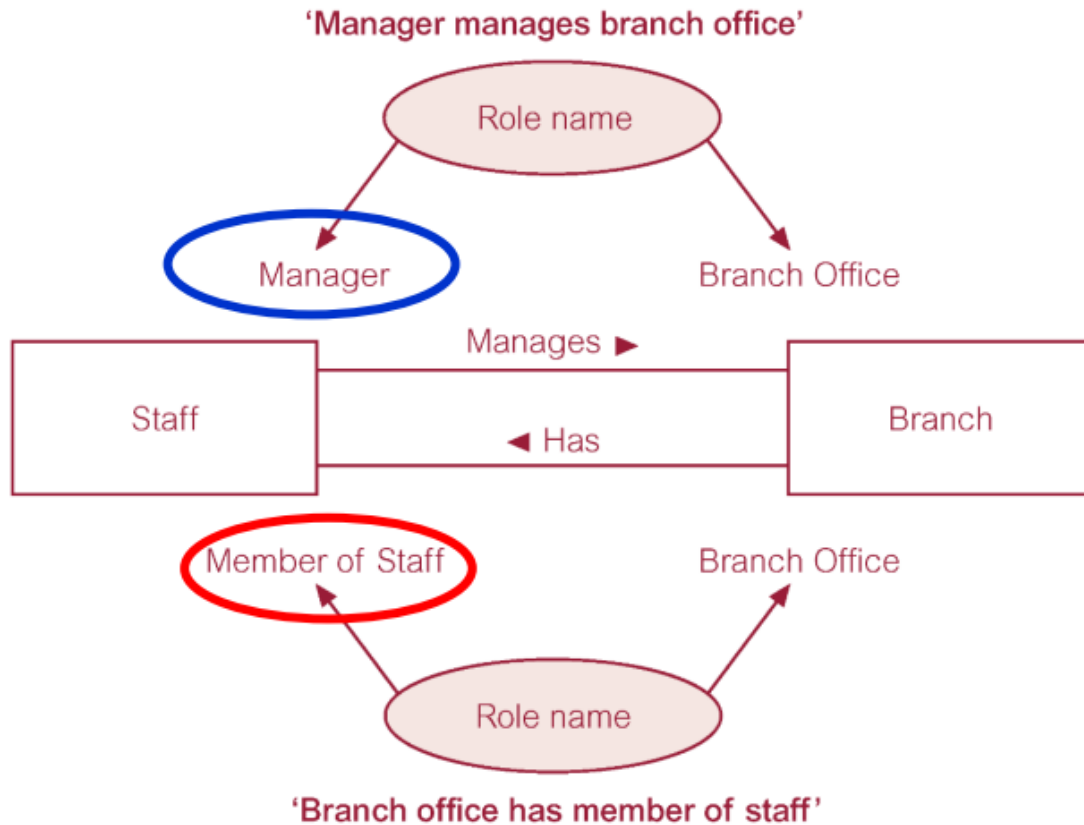
Role names are used to indicate the purpose each entity occurrence plays in the relationship.



1.2.4 Role names

Role names can also be used when the same entity participates multiple times in a relationship or participates in multiple relationships.

They clarify the purpose of each participating entity in the relationship.



1.3 Attributes

An **attribute** is a descriptive property of an entity or relationship.

An **attribute domain** is the set of allowable values for an attribute.

Attributes may be:

- simple or composite
 - e.g., **salary** or **address**
- single-valued or multi-valued
 - e.g., **staffID** or **telephoneNums**
- derived
 - e.g., **age** is derived from **dateOfBirth**
- null-valued
 - meaning the attribute currently has no value or the value is unknown/not applicable

1.3.1 Keys

Candidate key A **candidate key** is a minimal set of attributes whose values uniquely identify an entity occurrence. It cannot contain **null** values.

- minimal means that if any attribute is removed from the set, the remaining attributes no longer uniquely identify the entity occurrence.

A **simple** candidate key consists of a single attribute.

A **composite** candidate key consists of multiple attributes.

Primary key A **primary key** is the candidate key chosen to uniquely identify entity occurrences.

Factors considered when choosing a primary key include:

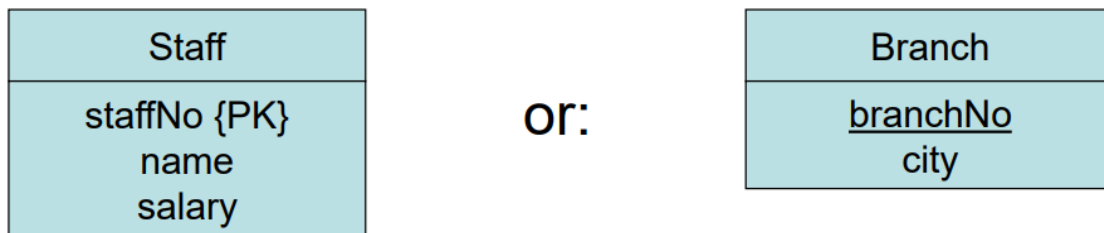
- number of attributes (preferably smaller)
- attribute length (preferably smaller)
- future certainty of uniqueness

1.3.2 Attributes on entity diagrams

Attributes are represented on ER diagrams by dividing the entity rectangle into two sections:

- the upper section contains the entity name
- the lower section contains the list of attributes

The primary key is usually: - underlined, or - labelled with the tag {PK}



1.3.3 Attributes on relationships

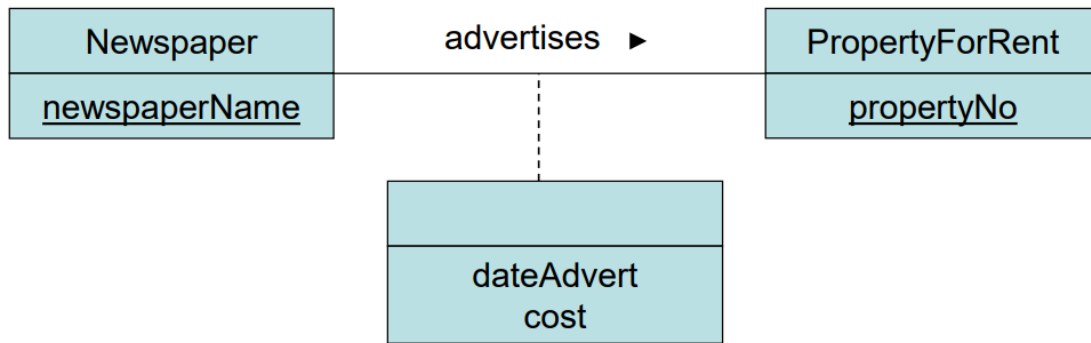
Relationships can also have attributes.

These describe properties of the relationship itself rather than properties of the participating entities.

To represent these, a labelled diagram with two parts is again used:

- the upper section is empty
- the lower section contains the list of relationship attributes

The rectangle is connected by a dashed line with the relationship.



1.4 Multiplicity

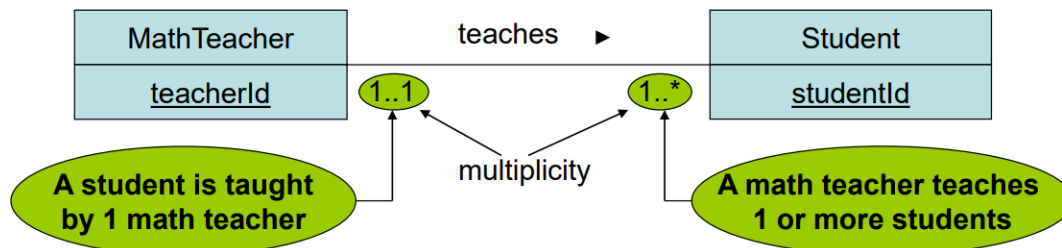
The multiplicity (or cardinality) of a relationship specifies the number of entity occurrences that can participate in the relationship.

Relationships can be:

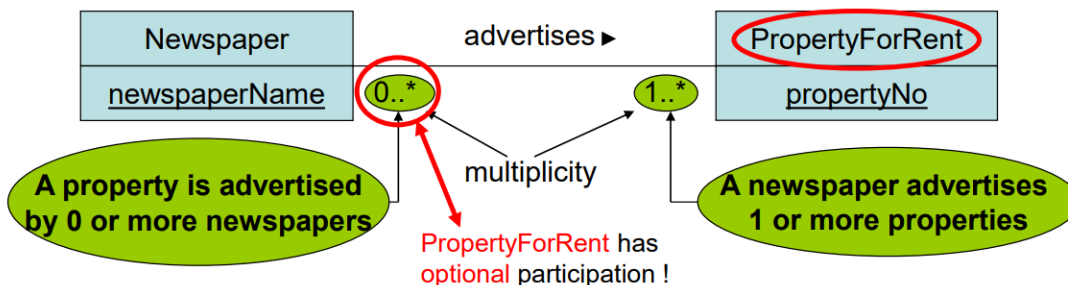
- one-to-one (1:1)
 - e.g., one **Staff** manages one **Branch**
- one-to-many (1:*)
 - e.g., one **MathTeacher** teaches many **Students**
- many-to-many (*:*)
 - e.g., Many **Newspapers** advertise many **PropertyForRent**

In an ER diagram, multiplicity constraints are shown on the relationship lines. They specify the minimum and maximum number of relationship occurrences in which an entity occurrence may participate.

example: (1:*) relationship:

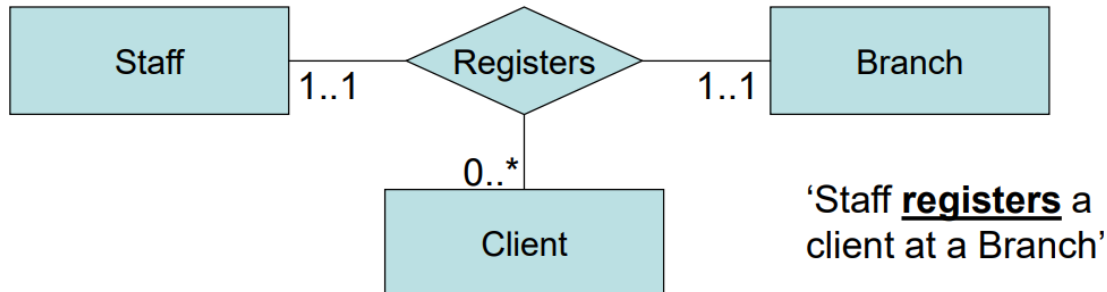


example: (*:*) relationship:



1.4.1 Multiplicity of n-ary relationships

For an n-ary relationship, multiplicity specifies the number of entity occurrences that may participate in the relationship when occurrences of the other $n - 1$ participating entity types are fixed.



In this example:

- for fixed **StaffNo** and **BranchNo**, there may be zero or more **ClientNo** values
 - each **Staff** member at a **Branch** may register 0 or more **Clients**
- for fixed **StaffNo** and **ClientNo**, there is exactly one **BranchNo** value
 - each **Staff** - **Client** pair is associated with exactly one **Branch**
- for fixed **ClientNo** and **BranchNo**, there is exactly one **StaffNo** value
 - each **Client** is registered at their **Branch** by exactly one **Staff** member

Therefore, the multiplicity of **Registers** is:

- with respect to **Staff** and **Branch**: 0..* (many)
- with respect to **Staff** and **Client**: 1..1 (one)
- with respect to **Client** and **Branch**: 1..1 (one)

1.5 Problems with the ER model

The basic concepts of the ER model (entities, attributes and relationships) are generally adequate for modelling simpler databases.

However, the ER model becomes limiting when modelling more complex database applications that require richer semantics and more advanced structural constraints.

For this reason, the Enhanced Entity-Relationship (EER) model was developed as an extension of the ER model.

1.6 The Enhanced Entity-Relationship (EER) model

The Enhanced Entity-Relationship (EER) model extends the ER model by introducing additional concepts that allow more expressive and intuitive database design.

1.6.1 Superclasses and subclasses

The EER model introduces the concepts of superclasses and subclasses:

- A **subclass** is a subset of an entity type whose occurrences share additional properties and require separate representation.

- A **superclass** is an entity type that has one or more distinct subclasses.

For example, the superclass **Staff** could have subclasses **Manager** and **Secretary**.

Each member of a subclass is also a member of the superclass.

Inheritance All attributes and relationships of the superclass are inherited by its subclasses.

A subclass may also have additional attributes and relationships that are not present in the superclass.

Type hierarchy A type hierarchy in the EER model is the organisation of entity types into a superclass-subclass structure in which subclasses inherit attributes and relationships from a more general superclass.

It can also be known as:

- Specialisation hierarchy
 - e.g., **Manager** is specialisation of **Staff**
- Generalisation hierarchy
 - e.g., **Staff** is generalisation of **Manager**
- IS-A hierarchy
 - e.g., **Manager** IS-A (member of) **Staff**

1.6.2 Advantages

The main advantages of the EER are that:

- similar concepts do not need to be repeatedly described, as common properties can be stored in a superclass and inherited by subclasses
- relationships can involve subclasses without involving the entire superclass, allowing more precise modelling
- it adds additional semantic information to the database design, making it closer to real-world meaning
 - e.g., **Manager** IS-A member of **Staff**, **Van** IS-A type of **Vehicle**

1.6.3 Specialisation

Specialisation is one approach to producing an EER model:

- it is a top-down process of maximising the differences between entity occurrences by identifying their distinguishing characteristics
- starting from one or more superclasses, it leads to the identification of more specific subclasses

1.6.4 Generalisation

Generalisation is the opposite approach:

- it is a bottom-up process of minimising differences between entity occurrences by identifying their common characteristics
- starting from subclasses, it leads to the formation of a more general superclass

1.6.5 Constraints

Participation constraint The participation constraint determines whether every entity in a superclass must belong to a subclass (total participation), or whether some entities may not belong to any subclass (partial participation).

It can be:

- mandatory (total specialisation): every superclass entity must belong to at least one subclass
- optional (partial specialisation): some superclass entities may not belong to any subclass

Disjoint constraint The disjoint constraint determines whether a superclass entity can belong to more than one subclass.

It only applies when there are at least two subclasses.

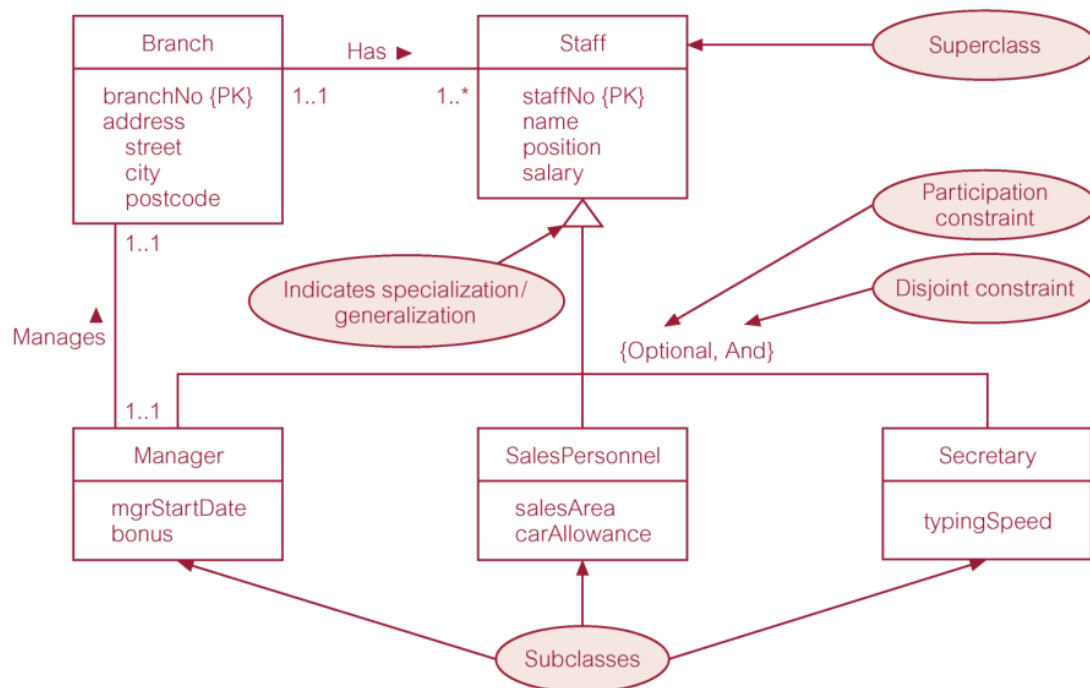
It can be:

- disjoint (or): an entity can belong to only one subclass
- overlapping (and): an entity can belong to more than one subclass

1.6.6 EER diagrams

In EER diagrams, specialisation/generalisation relationships are shown using a triangle (IS-A) symbol connecting the superclass and its subclasses.

- The triangle is labelled (or implied) as an IS-A relationship
- Connection lines from the subclasses meet at the triangle
- The triangle points towards the superclass
- Constraints are shown as a label near/below the triangle



2 Semi-structured data - XML

2.1 Types of data organisation

Data can be classified into three categories:

- structured data
- semi-structured data
- unstructured data

2.1.1 Structured data

Structured data is data that is represented in a strict format, defined by the schema.

- for example, in the relational data model (with tuples, tables and attributes)

The database management system (DBMS) enforces the schema to ensure that the data conforms to it, including:

- structure (tables, attributes, domains)
- integrity and referential constraints (primary keys and foreign keys)

2.1.2 Semi-structured data

Semi-structured data is data that:

- does not conform to a rigid, fixed schema (like relational tables), but
- still contains some form of structure within the data itself

It may be:

- irregular or incomplete
- flexible, with structure that can change over time
- heterogeneous (different data items may have different attributes)

This data may have some structure, but:

- not all the parts have the same fixed structure
- each data object may have different attributes that are not known in advance

Semi-structured data is often described as self-describing or schema-less data:

- the structure information is embedded within the data itself
- data and schema information are mixed together

In some cases, there may exist a separate schema, such as a Document Type Definition (DTD) or XML Schema Definition (XSD). These provide optional constraints but are not as rigid as relational schemas.

2.1.3 Unstructured data

Unstructured data has no predefined data model or strict structure. It does not follow a schema that can be easily represented in tables or fields

For example, a plain text document or image.

2.2 eXtensible Markup Language (XML)

eXtensible Markup Language (XML) is one of the most commonly used languages for representing semi-structured data:

- it is commonly used for storing, structuring and exchanging data, especially on the web

XML describes both the structure and content (semantics) of data.

2.3 Tags and terminology

XML uses tags (like HTML), which are enclosed using <> symbols.

Shown below is an example of some XML:

```
<bibliography>
  <book>
    <title> Foundations of Databases </title>
    <author> Abiteboul </author>
    <author> Hull </author>
    <author> Vianu </author>
    <publisher> Addison Wesley </publisher>
    <year> 1995 </year>
  </book>
  ...
</bibliography>
```

This example can be used to illustrate some key XML terminology:

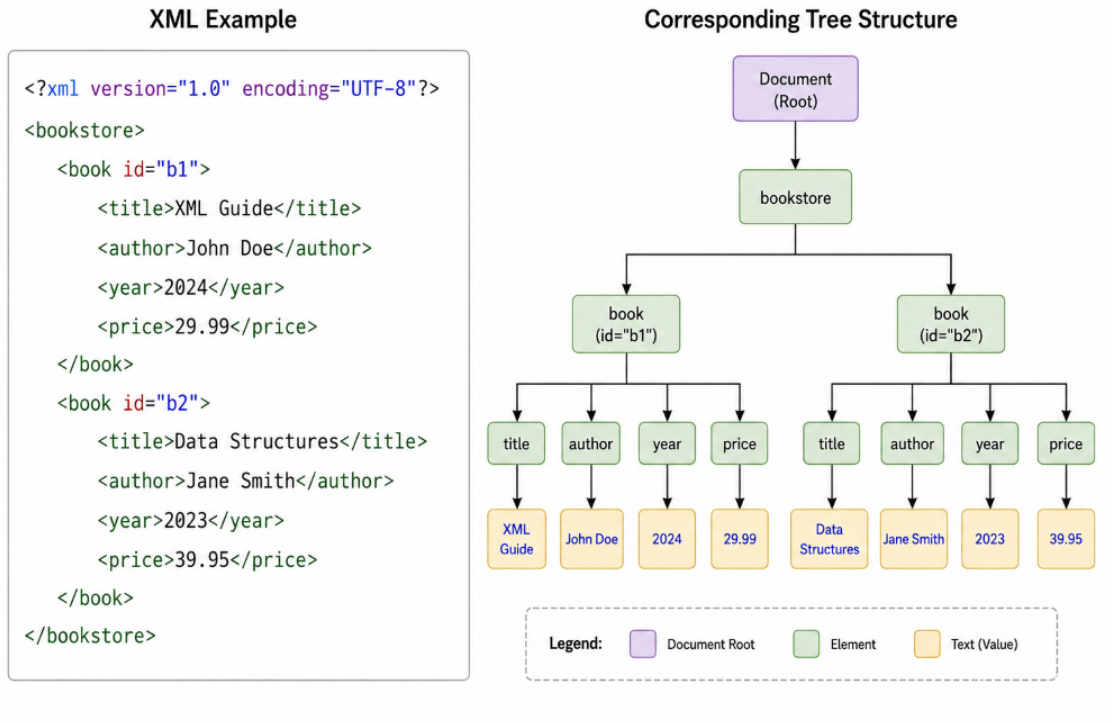
- tags: custom labels that describe what the data represents
 - for example, `book`, `title`, `author`
 - a start tag is enclosed in `<>`, for example `<book>`
 - an end tag is enclosed in `</>`, for example `</book>`
- elements: sections enclosed in start and end tags
 - for example, `<book> ... </book>`
 - elements may contain text, attributes or other nested elements
 - elements can be nested
 - elements can also be empty
 - * for example, `<author></author>` or `<author/>`
- root element: the single top-level element that contains all other elements
 - in this example, `bibliography`

An XML document must contain exactly one root element.

Note that XML is case-sensitive.

2.4 XML data as a graph

XML data can be represented as a directed tree structure.



In the tree:

- the document node is the root of the tree
- the root element of the XML is the child of the document node
- internal nodes are element nodes
- leaf nodes are text nodes (the actual data content)
- XML is order-sensitive: if two sibling elements are swapped, the structure and traversal order of the tree also changes

Query languages for XML navigate and query this tree-based representation.

2.5 Advantages of XML

There are many advantages of using XML to represent data:

- simplicity: it has a relatively simple, human-readable syntax
- extensibility: users can define their own tags for specific application needs
- platform- and vendor-independent: XML is widely supported and uses Unicode, enabling international character support
- separation of content and presentation: XML stores data independently of how it is displayed; presentation can be handled separately, allowing data to be reused in different formats

2.6 XML attributes

Attributes are a name-value pair with descriptive or identifying information about an element:

- they are placed inside the start tag of the element
- the attribute value is enclosed in quotes ""

- e.g., `<book id="b1">`
- attributes are associated with element nodes in the tree representation

2.6.1 Attributes vs elements

Attributes and elements are similar, but there are some important differences:

- an attribute name can appear only once within an element's start tag
- subelements can appear multiple times
 - subelements may contain the same attribute names
- attributes are unordered (subelements are ordered)
- attributes cannot contain nested structure (subelements can)
- attributes are generally less flexible and less extensible than elements
- attributes are typically used to store metadata (data about data)

2.6.2 ID and IDREF

Some XML attributes can be declared as specific types in a DTD:

- **ID**: used as a unique identifier for an element
- **IDREF**: used as a reference to an element with a specific ID
 - if an attribute is declared as **IDREF**, its value must match an existing ID
- **IDREFS**: an extension of **IDREF** that allows references to multiple elements
 - the value is a whitespace-separated list of ID values

The constraints associated with these types are defined and validated by the DTD (or schema).

ID and IDREF are similar to primary keys and foreign keys in the relational model.

When IDREF or IDREFS attributes are used, the XML structure is no longer purely a tree; it becomes a directed graph, which may contain cycles.

The example below shows some XML that uses ID and IDREF attributes

```
<person id="o555">
  <name>Jane</name>
</person>
<person id="o456" children-idrefs="o123 o555">
  <name>Mary</name>
</person>
<person id="o123" mother-idref="o456">
  <name>John</name>
</person>
```

2.7 More XML syntax

2.7.1 Comments

Comments in XML are enclosed in `<!--` and `-->` tags:

- they can contain any data except the string `--`
- they are ignored by XML parsers

2.7.2 CDATA

CDATA sections are used to include unparsed character data:

- can contain any text
- text inside CDATA is not treated as XML markup
- special characters (e.g. <, &) do not need to be escaped

2.7.3 Entity reference

Entity references are used to represent reserved or special characters in XML.

- they begin with & and end with ;
- during parsing, they are replaced with their corresponding character
 - e.g., & will be replaced with &

2.8 XML validation

Although XML is self-describing in terms of structure, additional constraints can be imposed to ensure consistency and correctness.

These constraints may define:

- permissible element names
- which elements can appear together
- how elements can be nested
- what attributes are allowed for each element type

2.8.1 Document Type Definition (DTD)

A Document Type Definition (DTD) is a formal set of rules that defines the structure of an XML document:

- specifies allowed elements and attributes
- defines how elements can be nested
- ensures consistent structure between XML documents
- appropriate for specific applications

The general structure of a DTD is:

```
<!DOCTYPE root_name [  
    <!ELEMENT elem_name (subElem1, subElem2, ...)>  
    ... more element declarations ...  

```

An element declaration defines the content model of an element:

- the element name (tag)
- the structure of its content (e.g. sequence, choice, repetition)
- e.g., <!ELEMENT book (title, author+)>
 - this means: a **book** contains a **title** followed by one or more **author** elements

#PCDATA stands for Parsed Character Data:

- it represents text content inside an element

- it is not a sub-element
- e.g., `<!ELEMENT title (#PCDATA)>`

Using the DTD To use the DTD, it can either be embedded directly within the XML file:

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE root_name [
    <!ELEMENT elem_name (subElem1, subElem2, ...)>
    ... more element declarations ...
]>
<root_name>
    ...
</root_name>
```

Or, the DTD is stored in a separate file and referenced using SYSTEM:

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE root_name SYSTEM "filename.dtd">
<root_name>
    ...
</root_name>
```

Sub-elements in a DTD Sub-elements are defined in the DTD content model and correspond to element names used in XML:

- sub-elements appear nested within an element in the XML document
- the order of sub-elements in XML must match the order specified in the DTD when using sequences
- commas define a sequence (ordered list) of sub-elements
- if commas are omitted, then they can appear in an arbitrary order
- the | symbol defines a choice between sub-elements

The multiplicity of a sub-element is specified by:

- *: zero or more
- +: one or more
- ?: = zero or one

Furthermore, | can be used to mean ‘or’, providing a choice between sub-element names.

Attributes in a DTD Attribute list declarations (`<!ATTLIST>`) define:

- which elements may have which attributes
- what types or values those attributes can take
- whether they are of type ID, IDREF, or IDREFS

DTD provides keywords to control whether attributes must appear:

- #REQUIRED: the attribute must always be provided
- #IMPLIED: the attribute is optional
- "value": specifies a default value if the attribute is omitted

For example:

```

<!ELEMENT book (title, author)>

<!--
  id          ID          #REQUIRED
  genre       CDATA       #IMPLIED
  language    CDATA       "English"
-->

```

In this example:

- `id` must always be provided
- `genre` is optional
- `language` defaults to "English" if omitted

2.8.2 XML schema

An XML Schema (XSD) is a more powerful and flexible alternative to DTD:

- supports complex structural constraints
- includes built-in data types
- written in XML syntax
- allows more precise validation rules than DTD

2.8.3 Well-formed XML

The first type of XML validation ensures that XML is well-formed:

- there is exactly one root element
- all elements are properly nested with matching start and end tags
- there is a valid XML declaration (optional)
 - e.g., `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>`
 - `standalone="yes"` indicates that the document does not depend on external declarations (such as external DTDs or entities)

2.8.4 Type-valid XML

The second type of XML validation ensures that XML is type-valid:

- the document must be well-formed
- elements and attributes must follow rules defined in a DTD

2.8.5 Schema-valid XML

The third type of XML validation ensures that XML is schema-valid:

- the document must be well-formed
- it must conform to an XML Schema (XSD)

3 XPath

XML Path Language (XPath) is a language used to navigate and select parts of an XML document.

It works by describing paths through the XML tree structure.

XPath:

- is a declarative query language for XML
- provides a simple syntax for addressing parts of an XML document
- treats an XML document as an ordered tree of nodes
- uses location paths composed of a series of steps joined with / (similar to directory paths)

3.1 XPath queries

For this section, the XML extract below will be used as an example:

```
<bib>
  <book>
    <publisher>Addison-Wesley</publisher>
    <author>Serge Abiteboul</author>
    <author>
      <firstName>Rick</firstName>
      <lastName>Hull</lastName>
    </author>
    <author>Victor Vianu</author>
    <title>Foundations of Databases</title>
    <year>1995</year>
  </book>

  <book price="55">
    <publisher>Freeman</publisher>
    <author>Jeffrey D. Ullman</author>
    <title>Principles of Database and Knowledge Base Systems</title>
    <year>1998</year>
  </book>
</bib>
```

3.1.1 Paths

An XPath expression selects the nodes matching a path through the XML tree.

- starting with / denotes an absolute path starting from the document root
- each / separates a navigation step

Example 1

/bib/book/year

selects

```
<year>1995</year>
<year>1998</year>
```

Example 2

/bib/paper/year

returns an empty result because there are no `paper` elements in the document.

3.1.2 Descendant search

Two forward slashes `//` can be used in an XPath expression to select matching descendant nodes at any depth in the XML tree

- it searches recursively through descendants of the current context node

Example 1

```
//author
```

selects

```
<author>Serge Abiteboul</author>
<author><firstName>Rick</firstName>
      <lastName>Hull</lastName>
</author>
<author>Victor Vianu</author>
<author>Jeffrey D. Ullman</author>
```

Example 2

```
/bib/book//firstName
```

selects

```
<firstName>Rick</firstName>
```

3.1.3 Wildcards

The wildcard symbol `*` matches any element node at a particular step in an XPath expression.

- it matches element nodes only (not text nodes or attributes)

Example 1

```
/bib/book/author/*
```

selects

```
<firstName>Rick</firstName>
<lastName>Hull</lastName>
```

because only one author element contains child elements.

Example 2

```
//book/*
```

selects

```
<publisher>Addison-Wesley</publisher>
<author>Serge Abiteboul</author>
<author>
  <firstName>Rick</firstName>
```

```

    <lastName>Hull</lastName>
</author>
<author>Victor Vianu</author>
<title>Foundations of Databases</title>
<year>1995</year>

<publisher>Freeman</publisher>
<author>Jeffrey D. Ullman</author>
<title>Principles of Database and Knowledge Base Systems</title>
<year>1998</year>

```

3.1.4 Attribute nodes

The @ symbol is used to select attributes:

- @attribute selects the attribute node named attribute
- @* selects all attributes of an element

Example

```
/bib/book/@price
```

selects the price attribute value:

```
55
```

3.1.5 Predicates

A predicate can be added after a node in an XPath expression:

- a predicate is an expression that evaluates to true or false
- it is written in square brackets [] after a node
- only nodes that satisfy the predicate are selected
- predicates often imply an existence test for an element or attribute

Example 1

```
//book[@price="55"]
```

selects all book elements with a price attribute equal to "55":

```

<book price="55">
  <publisher>Freeman</publisher>
  <author>Jeffrey D. Ullman</author>
  <title>Principles of Database and Knowledge Base Systems</title>
  <year>1998</year>
</book>

```

Example 2

```
/bib/book/author[firstName]
```

selects all author elements that contain at least one firstName child element:

```

<author>
  <firstName>Rick</firstName>
  <lastName>Hull</lastName>
</author>

```

3.1.6 Multiple predicates

A query can contain multiple predicates. Each predicate is written in square brackets [].

- predicates are applied from left to right
- each predicate filters the result of the previous step

If predicates are independent (i.e. they do not depend on position or different contexts), their order does not affect the final result.

Example

```
/bib//book[@price="55"][.//firstName | publisher = "Freeman"]
```

selects all book elements that:

- have a price attribute equal to "55"
- and either:
 - contain a firstName element anywhere inside them, or
 - have a publisher child element with value Freeman

```

<book price="55">
  <publisher>Freeman</publisher>
  <author>Jeffrey D. Ullman</author>
  <title>Principles of Database and Knowledge Base Systems</title>
  <year>1998</year>
</book>

```

Since these two predicates are independent, they can be reversed without changing what is selected.

3.1.7 Axes

The general form of an XPath query is

```
/step-1/step-2/step-3/.../step-n
```

Each of these steps consists of:

- a basis, and
- a list of predicates (optional)

A basis consists of:

- an axis: the direction in which the navigation proceeds from the current node
- a node test: the type or name of node to navigate to

The default axis is:

```
child::
```

So expressions like:

`/bib/book`

are equivalent to:

`child::bib/child::book`

There are many other axes, such as:

- `parent::` - selects the parent of the current node
- `attribute::` - selects attributes of the current node
- `self::` - selects the current node itself
- `child::` - selects child elements (default axis)
- `descendant::` - selects all descendants of the current node
- `descendant-or-self::` - selects the current node and all descendants
- `ancestor::` - selects all ancestors of the current node
- `ancestor-or-self::` - selects ancestors and the current node
- `following-sibling::` - nodes with the same parent that come after the current node
- `preceding-sibling::` - nodes with the same parent that come before the current node
- `following::` - all nodes in document order after the current node
- `preceding::` - all nodes in document order before the current node

Syntax An axis for a tag (or attribute) is selected by prefixing it with an axis name and a double colon `::`.

Example 1

`/bib/book/@price`

is equivalent to:

`/child::bib/child::book/attribute::price`

Example 2

`//book/*`

is equivalent to:

`/descendant-or-self::book/child::*`

3.1.8 Functions

Two key XPath functions are:

- `text()`: selects the text node children of an element
- `name()`: returns the name of the current node as a string

Example 1

`/bib/book/author/text()`

selects

Serge Abiteboul
Victor Vianu
Jeffrey D. Ullman

Example 2

```
/bib//*[name() = "book"]
```

is equivalent to `/bib//book` and selects

```
<book>
  <publisher>Addison-Wesley</publisher>
  ...
</book>
<book price="55">
  <publisher>Freeman</publisher>
  ...
</book>
```

3.1.9 Positional predicates

The `position()` function returns the position of a node within the current node set being evaluated.

It is used to select nodes based on their position relative to sibling nodes.

Example

```
//author[position() = 2]
```

selects the second `author` child element of each parent node:

```
<author> <firstName>Rick</firstName>
        <lastName>Hull</lastName>
</author>
```

The same result can be written more concisely as:

```
//author[2]
```

4 XQuery

XQuery is a powerful query and transformation language for XML.

It is often described as the SQL equivalent for XML data.

It:

- uses XPath to navigate XML data
- selects and filters nodes
- combines and transforms data
- constructs new XML or other output formats

4.1 Basic query form

The basic form of an XQuery query is a FLWR expression:

```
FOR ...
```

```
LET ...
```



```
WHERE ...
RETURN ...
```

The purpose of each clause is as follows:

- **FOR**: iterates over a sequence of nodes
- **LET**: binds a sequence to a variable
- **WHERE**: filters nodes based on a condition
- **RETURN**: constructs the output

4.1.1 FOR-WHERE-RETURN

A **FOR-WHERE-RETURN** expression iterates over nodes, filters them, and returns results.

Example

```
FOR $x IN doc("bib.xml")/bib/book
WHERE $x/year > 1995
RETURN { $x/title }
```

In this query:

- **FOR** `$x IN doc("bib.xml")/bib/book` iterates over each `book` element selected by the XPath expression `doc("bib.xml")/bib/book`, and binds it to the variable `$x`
- **WHERE** `$x/year > 1995` filters books whose `year` value is greater than 1995
- **RETURN** `{ $x/title }` returns the `title` element of each selected book

Continuing with the [example XML from earlier](#), the result would be:

```
<title>Database System Implementation</title>
<title>Introduction to Databases</title>
```

This query could be written equivalently in XPath as

```
doc("bib.xml")/bib/book[year > 1995]/title
```

4.1.2 FOR vs LET

FOR binds a variable to each item in a sequence one at a time:

- it iterates over a sequence of nodes
- the variable is rebound for each item

For example

```
FOR $x IN doc("bib.xml")/bib/book
```

means `$x` takes each `book` element one by one during iteration.

LET binds a variable to an entire sequence at once:

- no iteration occurs
- the variable stores the full collection

For example

```
LET $x := doc("bib.xml")/bib/book
```

means `$x` contains the full sequence of all `book` elements.

4.1.3 Collections

A collection is a sequence of items bound to a variable using `LET`.

- sequences are ordered by default
- sequences may contain duplicate values or nodes

The function:

`distinct-values(...)`

is used to remove duplicate atomic values from a sequence.

For example,

`distinct-values(/bib/book/author/text())`

returns a sequence of unique author names.

4.1.4 WHERE

The `WHERE` clause contains one or more conditions used to filter results produced by the `FOR` and `LET` clauses.

For variables bound by a `FOR` clause:

- the variable is evaluated one item at a time
- predicates are typically applied to individual elements or values
- commonly used with scalar conditions
 - e.g., `$x/salary > 10000`

For variables bound by a `LET` clause:

- the variable usually contains an entire sequence
- predicates are often applied to the sequence as a whole
- commonly used with aggregate or list-oriented functions
 - e.g., `avg($S/SALARY) > 10000`

Common aggregate functions include:

- `count()`: returns the number of items in a sequence
- `avg()`: returns the average of numeric values in a sequence
- `min()`, `max()`: return the minimum and maximum values respectively
- `sum()`: returns the sum of numeric values in a sequence
- `string-length()`: returns the length of a string
- `empty()`: checks whether a sequence is empty
- `exists()`: checks whether a sequence contains at least one item

In XPath/XQuery comparison operations, automatic type processing is performed:

- atomization: converts nodes (such as elements) into their atomic values
- casting: converts atomic values into an appropriate type for comparison (e.g. integer, string)

4.1.5 Joins

XQuery supports joins between sequences of XML nodes, similar to joins in SQL.

For example:

```
FOR $S IN doc("staff_list.xml")//Staff,  
    $B IN doc("branch.xml")//Branch  
WHERE $S/branchNo = $B/branchNo  
RETURN
```

```
    <staff-branch>  
      {$S}  
      {$B/Address}  
    </staff-branch>
```

- the query iterates over all Staff and Branch elements
- the WHERE clause matches staff members to branches with the same **branchNo**
- the RETURN clause constructs a new XML element combining the matched data

4.1.6 Ordering

The ORDER BY clause is used to control the order in which query results are returned:

- it sorts the results of a FLWR expression
- sorting is usually based on element or attribute values
- ascending order is the default
- descending can be specified explicitly

For example,

```
FOR $b IN doc("bib.xml")/bib/book  
ORDER BY $b/year  
RETURN $b/title
```

returns book titles ordered by year.

4.1.7 IF-THEN-ELSE

XQuery supports conditional branching using IF-THEN-ELSE expressions:

```
FOR $h IN doc("bibliography.xml")//papers  
ORDER BY $h/title  
RETURN
```

```
    <holding>  
      {  
        IF ($h/@type = "Journal")  
        THEN $h/editor  
        ELSE $h/author[1]  
      }  
    </holding>
```

In this example:

- if the **type** attribute of the current paper is "Journal", then the **editor** element is returned

- otherwise, the first `author` child element is returned

4.1.8 Existential qualifiers: `SOME`

The `SOME` keyword is used to test whether at least one item in a sequence satisfies a condition.

```
FOR $b IN doc("bibliography.xml")//book
WHERE SOME $p IN $b//chapter SATISFIES
    (
        contains($p, "sailing")
        AND $p/length < 40
    )
RETURN {$b/title}
```

This query returns the `title` of each `book` where at least one `chapter`:

- contains the word `"sailing"`
- and has a `length` value less than 40

4.1.9 Universal qualifiers: `EVERY`

The `EVERY` keyword is used to test whether all items in a sequence satisfy a condition.

```
FOR $b IN doc("bibliography.xml")//book
WHERE EVERY $p IN $b//chapter SATISFIES
    (
        contains($p, "sailing")
        AND $p/length < 40
    )
RETURN {$b/title}
```

This query returns the `title` of each `book` where every `chapter`:

- contains the word `"sailing"`
- and has a `length` value less than 40

5 Transactions

5.1 Database Management Systems

A Database Management System (DBMS) is software used to create, store, manipulate, and manage databases efficiently.

A DBMS must ensure that a database:

- is reliable
 - operations behave correctly and as expected
- remains consistent
 - the database does not enter an invalid or contradictory state

To achieve this, the DBMS must be able to handle:

- software and hardware failures
- multiple users accessing the database concurrently

These issues are managed using:

- database recovery
 - the process of restoring the database to a correct and consistent state after a failure
- concurrency control protocols
 - mechanisms that prevent simultaneous transactions from interfering with one another

To support recovery and concurrency control, DBMSs use the concept of transactions.

5.2 Transactions

A **transaction** is a sequence of one or more database operations performed as a single logical unit of work by a user or program

Transactions may:

- read data
- modify data
- insert data
- delete data

A transaction usually represents a single real-world task, such as transferring money between bank accounts.

At the end of a transaction:

- the database must be in a consistent state
 - all integrity constraints (such as primary key and foreign key constraints) are satisfied

During execution of a transaction:

- the database may temporarily be in an inconsistent state
 - some constraints may be temporarily violated until the transaction completes

A transaction can have two outcomes:

- committed
 - the transaction completed successfully, so all changes are permanently saved to the database
- rolled back (or aborted)
 - the transaction failed or was cancelled, so all changes made by the transaction are undone

5.2.1 Properties of transactions

All transactions must satisfy the ACID properties:

- atomicity
- consistency
- isolation
- durability

Atomicity Atomicity means that a transaction is performed entirely or not performed at all.

If any part of the transaction fails, the entire transaction is rolled back and the database is restored to its previous state.

The responsibility for atomicity lies with the recovery subsystem of the DBMS.

Consistency Consistency means that a transaction must transform the database from one consistent state to another consistent state.

After a transaction completes, all integrity constraints (such as primary key and foreign key constraints) must still hold.

Responsibility for consistency lies with both the DBMS and the application developers.

- for example, the DBMS can enforce integrity constraints, but incorrect transaction logic written by a programmer may still produce invalid data

Isolation Isolation means that concurrently executing transactions do not interfere with one another.

- the intermediate (partial) effects of an incomplete transaction should not be visible to other transactions

From the perspective of each transaction, it should appear as though it is executing alone on the database.

Responsibility for isolation lies with the concurrency control subsystem of the DBMS.

Durability Durability means that once a transaction has been committed, its changes are permanently stored and will not be lost due to failures such as software crashes or power loss.

Responsibility for durability lies with the recovery subsystem of the DBMS.

5.3 Concurrency control

Concurrency control is the process of managing simultaneous operations on a database so that transactions do not interfere with one another.

Two transactions may each be correct when executed independently, but if they execute concurrently, they may produce an inconsistent database state.

Common problems caused by concurrent transactions include:

- lost update problem
- uncommitted dependency problem
- inconsistent analysis problem

5.3.1 Lost update problem

The lost update problem occurs when two transactions both update the same data item and one update overwrites the other.

As a result, one transaction's update is effectively lost, even though both transactions completed successfully.

Time	Transaction T1 (Withdraw 100)	Transaction T2 (Withdraw 200)
t1	READ balance → 1000	
t2		READ balance → 1000
t3	balance = 1000 - 100 = 900	
t4		balance = 1000 - 200 = 800
t5	WRITE balance → 900	
t6		WRITE balance → 800
t7		
t8	Final balance in database = 800 (The update of T1 is lost)	

5.3.2 Uncommitted dependency problem

The uncommitted dependency problem occurs when a transaction is allowed to read data that has been modified by another transaction that has not yet committed.

If the other transaction is later rolled back, the first transaction will have used invalid data.

This problem is also known as a dirty read.

Time	Transaction T1 (Update and Rollback)	Transaction T2 (Read)
t1	READ balance → 1000	
t2	WRITE balance → 500 (UNCOMMITTED)	
t3		READ balance → 500 (DIRTY READ)
t4	Something goes wrong... ROLLBACK	
t5		Use balance = 500 for further processing
t6		
t7	Final balance in database = 1000 (because T1 rolled back) But T2 read and used an invalid value (500).	

5.3.3 Inconsistent analysis problem

The inconsistent analysis problem occurs when a transaction reads a set of data while another transaction is updating part of that data.

As a result, the transaction may see some old values and some new values, producing inconsistent results.

This commonly occurs when one transaction is performing an aggregate operation such as SUM or COUNT while another transaction updates the same data.

Time	Transaction T1 (Compute Total)	Transaction T2 (Update)
t1	READ A → 100	
t2		UPDATE B: 200 → 300 COMMIT
t3	READ B → 300	
t4		UPDATE C: 300 → 400 COMMIT
t5	READ C → 400	
t6	Compute Total = 100 + 300 + 400 = 800	
t7	T1's total = 800 (inconsistent snapshot) Actual total before any update was $100 + 200 + 300 = 600$ Actual total after all updates is $100 + 300 + 400 = 800$	

5.4 Schedules

An obvious solution to concurrency problems is to allow only one transaction to execute at a time.

- i.e. one transaction completes (commits or rolls back) before the next transaction begins

However, this approach greatly reduces concurrency and system performance.

To maximise concurrency while still maintaining database consistency, the DBMS uses concurrency control mechanisms.

One important concept used in concurrency control is a schedule.

One such mechanism is schedules:

- a **schedule** is an ordering of the operations from a set of concurrent transactions $T_1, T_2, \dots, T_n$ such that the order of operations within each individual transaction is preserved.
- a **serial schedule** is a schedule in which the operations of different transactions are not interleaved
 - one transaction executes completely before another begins
- a **non-serial schedule** is a schedule in which operations from different transactions are interleaved

5.4.1 Serialisable schedules

Any serial schedule leaves the database in a consistent state, although different serial schedules may produce different valid database states.

A non-serial schedule is serialisable if it produces the same final database state as some serial schedule involving the same transactions.

In other words, a serialisable schedule behaves equivalently to a serial execution of those transactions.

Operation conflicts Two operations are not in conflict if:

- they are both read operations on the same data item
- they operate on different data items

Two operations are in conflict if they belong to different transactions, operate on the same data item, and at least one of them is a write operation.

In serialisability, only the ordering of conflicting operations matters.

- non-conflicting operations may be reordered without affecting the correctness of the schedule

Conflict serialisability Conflict serialisability ensures that a concurrent schedule produces the same effect as some serial execution of the same transactions.

To determine whether a non-serial schedule is conflict serialisable, a precedence graph (or serialisation graph) can be constructed.

A precedence graph is a directed graph $G = (V, E)$, where:

- each node $T_i \in V$ represents a transaction
- a directed edge $T_i \rightarrow T_j$ exists if:
 - an operation in T_i conflicts with an operation in T_j , and
 - the operation in T_i occurs before the conflicting operation in T_j in the schedule

In the precedence graph, an edge $T_i \rightarrow T_j$ means that T_i must appear before T_j in any equivalent serial schedule.

A schedule is conflict serialisable if and only if its precedence graph contains no directed cycles.

View serialisability View serialisability is another form of serialisability based on whether transactions observe the same values.

Two schedules S_1 and S_2 are view equivalent if all of the following conditions hold:

- for each data item x , if transaction T_i reads the initial value of x in S_1 , then T_i also reads the initial value of x in S_2
- for each read operation $r_i(x)$ in S_1 , if transaction T_i reads the value of x written by transaction T_j , then in S_2 , T_i must also read the value of x written by T_j
- for each data item x , if transaction T_i performs the final write on x in S_1 , then T_i must also perform the final write on x in S_2

In other words, two schedules are view equivalent if:

- the same transactions read the same values, and

- the same transaction performs the final write on each data item

A non-serial schedule is view serialisable if it is view equivalent to some serial schedule.

Every conflict serialisable schedule is also view serialisable. However, the converse is not always true.

5.5 Concurrency control approaches

There are two main approaches used to ensure consistency when transactions execute concurrently:

- conservative (pessimistic) methods
 - assume that conflicts between transactions are likely
 - actively prevent conflicts from occurring
 - delay, block, or restart transactions when conflicts are detected
- optimistic methods
 - assume that conflicts between transactions are rare
 - allow transactions to execute freely without immediate restrictions
 - check for conflicts only before commit (during validation)

5.6 Locking

Locking is a conservative (pessimistic) approach to concurrency control.

A **lock** is a mechanism used by the DBMS to control concurrent access to data items and prevent transactions from interfering with one another.

A transaction T can keep two types of locks:

- shared lock (or read lock):
 - T is allowed to read the data item
 - other transactions may also acquire shared locks on the same item and read it
 - no transaction may write to the item while shared locks exist
- exclusive lock (or write lock):
 - T is allowed to read and write the data item
 - no other transaction may read or write the item while the exclusive lock is held

Locks are used as follows:

- a transaction requests a lock from the DBMS
- if the lock is compatible with existing locks, the DBMS grants it
- otherwise, the transaction is blocked (waits) until the lock can be granted
- when the transaction no longer needs the data item, it releases the lock

5.6.1 Rules for locks

When a transaction needs to access a data item, it requests either: - a shared lock for read-only access, or - an exclusive lock for read and write access

Then: - if no other transaction holds an incompatible lock on the item, the requested lock is granted
 - otherwise, the DBMS checks whether the requested lock is compatible with the existing lock(s): - if the item currently has only shared locks and another shared lock is requested, the lock is granted
 - otherwise, the requesting transaction must wait until the conflicting lock is released

A transaction holds a lock until it explicitly releases it:

- during execution, once the item is no longer needed, or
- when the transaction terminates (commits or aborts)

Exclusive locks prevent other transactions from reading or writing the locked item while the transaction is updating it.

This helps ensure the Isolation property of transactions.

To improve concurrency, some systems may allow locks to be:

- upgraded from a shared lock to an exclusive lock, or
- downgraded from an exclusive lock to a shared lock

5.6.2 Two-phase locking

Locking alone does not guarantee serialisability.

To guarantee conflict serialisability, an additional locking protocol called two-phase locking (2PL) is used.

In two-phase locking, each transaction has two distinct phases:

- growing phase
 - the transaction may acquire locks
 - no locks may be released
- shrinking phase
 - the transaction may release locks
 - no new locks may be acquired

Therefore, once a transaction releases its first lock, it cannot obtain any additional locks.

If every transaction in a schedule follows the two-phase locking protocol, then the schedule is guaranteed to be conflict serialisable.

5.6.3 Cascading rollback problem

Although 2PL guarantees conflict serialisability, problems can still occur if locks are released before a transaction commits.

This can lead to the cascading rollback problem.

A cascading rollback occurs when one transaction reads or uses data written by another uncommitted transaction that later aborts.

As a result, the dependent transaction must also be rolled back, potentially causing a chain of rollbacks.

For example:

- T_1 obtains an exclusive lock on data item x , updates x , and releases the lock before committing
- T_2 then accesses and updates the modified value of x
- $T_3, T_4, \dots, T_n$ continue using values derived from previous uncommitted transactions
- later, T_1 aborts and its update to x is undone
- because T_2 used the invalid value written by T_1 , T_2 must also be rolled back

- T_3 must then be rolled back because it depended on T_2 and so on

This chain reaction of dependent rollbacks is called a cascading rollback.

5.6.4 Rigorous two-phase locking

Rigorous two-phase locking is a stricter version of the two-phase locking protocol.

With rigorous two-phase locking:

- all locks (both shared and exclusive) are held until the transaction either commits or aborts
- no locks are released during the execution of the transaction

This guarantees:

- serialisability
- no cascading rollbacks
- schedules are equivalent to a serial order based on commit order

Strict two-phase locking is a variant of this, where all exclusive locks are released at the end of the transaction, but shared locks can be released earlier.

5.7 Deadlocks

Deadlocks are a problem that can occur when using locking-based concurrency control.

A deadlock occurs when two or more transactions are each waiting for a lock held by another, forming a cycle of waiting. As a result, they may wait indefinitely.

For example:

- T_1 gets the exclusive lock on x
- T_2 gets the exclusive lock on y
- T_1 requests an exclusive lock on y
- T_2 requests an exclusive lock on x
- neither request can be granted, so both transactions must wait
- each transaction is waiting for the other to release a lock, forming a circular wait (deadlock)

5.7.1 Handling deadlocks

One common method for handling deadlocks is timeouts:

- a transaction requests a lock and is only allowed to wait for a system-defined maximum time
- if the lock is not granted within this time:
 - the DBMS assumes there is a deadlock
 - the transaction request times out
 - the transaction rolls back and restarts

Another method is deadlock detection:

- the DBMS constructs a wait-for graph $G = (N, E)$ where:
 - each node $T_i \in N$ represents a transaction
 - a directed edge $T_i \rightarrow T_j$ exists if T_i is waiting for a lock currently held by T_j
- a deadlock exists if and only if the wait-for graph contains a directed cycle
- when a deadlock is detected:

- the DBMS selects one (or more) transactions as a victim
- the victim transaction is aborted (rolled back) to break the cycle

5.8 Recoverable schedules

A recoverable schedule is a schedule that ensures that once a transaction has committed, it will never need to be rolled back.

This guarantees the **durability** of transactions:

- once a transaction has been committed, its changes are permanently stored

A schedule S is recoverable if:

- whenever a transaction T_j reads a value written by another transaction T_i then
- T_j is not allowed to commit until after T_i has committed

In other words, transactions may only commit after all transactions whose data they have used have committed first.

This prevents a committed transaction from depending on data written by a transaction that later aborts.

5.9 Cascadeless schedules

A cascadeless schedule is a schedule in which cascading rollbacks cannot occur.

A schedule is cascadeless if:

- whenever a transaction T_j reads a data item written by T_i then
- T_i has already committed before the read operation of T_j

In other words, transactions are only allowed to read committed data.

This prevents transactions from depending on uncommitted updates that may later be rolled back.

There is an important difference between cascadeless schedules and recoverable schedules:

- in a cascadeless schedule, transactions only read committed data
 - therefore, cascading rollbacks can never occur
- in a recoverable schedule, transactions may read uncommitted data -however, they are not allowed to commit until the transaction they depend on has committed therefore, cascading rollbacks are still possible

Every cascadeless schedule is recoverable, but not every recoverable schedule is cascadeless.

5.10 Schedules and consistency

To ensure correct concurrent execution of transactions, schedules should be:

- serialisable
 - the schedule produces the same effect as some serial execution of the same transactions
 - this preserves database consistency
- recoverable
 - once a transaction commits, it will never need to be rolled back
 - this preserves reliability and supports durability

Ideally, schedules should also be:

- cascadeless
 - transactions only read committed data
 - this prevents cascading rollbacks and improves efficiency

5.11 Timestamping

Timestamping is a conservative (pessimistic) approach to concurrency control (it actively tries to prevent conflicts from occurring).

It ensures conflict serialisability by enforcing a global order of transaction execution based on timestamps. Conflicting operations that violate the timestamp order are rejected, causing transactions to be rolled back and restarted.

Timestamping ordering works as follows:

- each transaction T_i is assigned a unique timestamp $TS(T_i)$
 - the timestamp represents the relative start time of the transaction
 - the DBMS may generate timestamps:
 - * using the system clock, or
 - * by incrementing a logical counter for every new transaction
- timestamps determine the equivalent serial order of transactions
 - if $TS(T_i) < TS(T_j)$, then T_i must appear before T_j in the equivalent serial schedule

For each data item x , the DBMS maintains: - $WTS(x)$: writing time stamp - the largest timestamp of any transaction that successfully wrote x - $RTS(x)$: reading time stamp - the largest timestamp of any transaction that successfully read x

The protocol ensures that conflicting operations occur in timestamp order.

5.11.1 Read rule

Suppose transaction T tries to read data item x :

- if $TS(T) < WTS(x)$ then
 - a newer transaction has already written x
 - allowing T to read x would violate the timestamp order
 - therefore, the read operation is rejected and T is rolled back
- otherwise, $TS(T) \geq WTS(x)$, so
 - the read operation is executed
 - $RTS(x)$ is set to the maximum of $TS(T)$ and the current value for $RTS(x)$

5.11.2 Write rule

Suppose transaction T tries to write data item x :

- if $TS(T) < RTS(x)$ then
 - a newer transaction has already read x
 - allowing T to write x would violate the timestamp order
 - therefore, T is rolled back
- else if $TS(T) < WTS(x)$ then
 - a newer transaction has already written x

- therefore, the write is rejected and T is rolled back
- otherwise,
 - the write operation succeeds
 - $WTS(x)$ is updated to $TS(T)$

5.11.3 Correctness of timestamping

The timestamp ordering protocol guarantees conflict serialisability, because all conflicting operations are executed in timestamp order.

In the precedence graph:

- all edges go from a transaction with a smaller timestamp to a transaction with a larger timestamp
- therefore, cycles cannot occur
- hence the schedule is conflict serialisable

The timestamp method also ensures freedom from deadlock, since transactions do not wait (they are either executed or rolled back).

However:

- timestamp ordering does not guarantee recoverability
- it may produce schedules that allow cascading rollbacks

5.11.4 Recoverability in timestamping

Suppose T_i writes a data item x , and T_j reads x .

If T_i later aborts:

- T_j must also be rolled back, because it read invalid data

If T_j has already committed before T_i aborts: - the schedule is not recoverable, because a committed transaction depended on uncommitted data

The solution to this is as follows:

- a transaction is structured such that all its writes are all performed at the end of its processing
- all writes of a transaction form an atomic action; no transaction may execute while a transaction is being written
- a transaction that aborts is restarted with a new timestamp

5.11.5 Thomas' write rule

Thomas' write rule is an optimisation of the timestamp ordering protocol that improves concurrency by ignoring obsolete write operations.

It produces schedules that are equivalent to timestamp-ordered execution, and may allow schedules that are not conflict serialisable, but are still correct under the timestamp ordering model.

- Conflict serialisable schedules:
 - can be transformed into a serial schedule by swapping non-conflicting operations
- View serialisable schedules:
- preserve:

- which transaction reads which values
- the final write on each data item

Thomas' write rule allows more schedules than strict timestamp ordering by safely ignoring certain writes.

The ignore obsolete write rule (Thomas' write rule) is:

- when a transaction T tries to write data item x :
 - if $TS(T) < RTS(x)$ then:
 - * the write is rejected
 - * T is rolled back (same as basic timestamp ordering)
 - else if $TS(T) < WTS(x)$ then:
 - * the write is ignored (it is obsolete)
 - otherwise:
 - * the write is executed
 - * $WTS(x) = TS(T)$

This works because:

- if $TS(T) < WTS(x)$ then:
 - a newer transaction has already written x
 - the system state already reflects a more recent value
 - therefore, T 's write is obsolete and would never affect any future correct schedule
 - so it can be safely ignored without affecting correctness

5.12 Multiversion timestamps

Multiversion timestamping is an extension of the timestamp ordering protocol that increases concurrency by allowing multiple versions of a data item.

- multiple transactions can access the same data item x simultaneously
- each transaction reads/writes a version of x
- each version is tagged with timestamps
- this reduces conflicts and increases concurrency

5.12.1 Data structure

Each data item x has multiple versions $x_1, x_2, \dots, x_n$.

Each version x_k stores:

- the value of x_k
- $WTS(x_k)$: timestamp of the transaction that created x_k
- $RTS(x_k)$: largest timestamp of any transaction that read x_k

When a new version is created by transaction T :

- $WTS(x_k)$ and $RTS(x_k)$ are both initialised to $TS(T)$

5.12.2 Read operation

When a transaction T reads x :

- the version x_k with the largest $WTS(x_k) \leq TS(T)$ is returned

A read is always successful.

5.12.3 Writing in multiversion timestamps

When a transaction T requests to write x :

- let x_k be the version of x with largest $WTS(x_k) \leq TS(T)$
- if $TS(T) < RTS(x_k)$:
 - T is rolled back
 - a newer transaction has already read this version
 - rewriting would break serialisability
- if $TS(T) \geq RTS(x_k)$ then:
 - if $TS(T) = WTS(x_k)$:
 - * the contents of x_k are overwritten
 - * x_k was created by the same transaction T
 - if $TS(T) > WTS(x_k)$:
 - * a new version of x is created

5.12.4 Version deletion

Old versions can be removed when they are no longer needed.

To determine this:

- find the smallest timestamp TS_{min} among all active (uncommitted) transactions
- delete all x_k $WTS(x_k) < TS_{min}$ besides the one with the largest $WTS(x_k)$
 - this version is kept in case the oldest transaction is rolled back

5.13 Optimistic concurrency control

Optimistic concurrency control assumes that transaction conflicts are rare.

Therefore:

- transactions are allowed to execute freely without locking
- conflicts are checked only near the end of the transaction
 - typically just before commit
- if a conflict is detected, the transaction is aborted and restarted

Optimistic concurrency control is generally more efficient when conflicts occur infrequently.

Optimistic concurrency control typically consists of three phases:

- read phase
- validation phase
- write phase (only for write operations)

5.13.1 Read phase

The read phase runs from the beginning of the transaction until just before validation.

During this phase:

- required data items are read from the database
- values are stored in local variables (local workspace copies)
- all updates are applied only to the local copies
- the actual database is not modified yet

Thus, transactions can execute without interfering with each other during the read phase.

5.13.2 Validation phase

The validation phase occurs after the read phase.

During validation, the DBMS checks whether allowing the transaction to commit would violate serialisability.

If validation fails:

- the transaction is aborted
- its local changes are discarded -the transaction may be restarted

For read-only transactions:

- validation checks whether the values read are still valid
- if no conflicting updates occurred, the transaction commits
- otherwise, the transaction aborts and restarts

For transactions that modify data (i.e., write): - validation checks whether applying the updates would maintain: - database consistency - serialisability - if validation succeeds, the transaction proceeds to the write phase - otherwise, the transaction is aborted and restarted

5.13.3 Write phase

The write phase occurs only after successful validation.

During this phase:

- the updates stored in the local workspace are written to the database
- the transaction commits

5.14 Validation

In optimistic concurrency control, each transaction T is assigned three timestamps:

- $\text{start}(T)$
: the time when T begins execution (start of the read phase)
- $\text{validation}(T)$: the time when T begins validation
- $\text{finish}(T)$: the time when T completes its write phase

Validation determines whether committing T would violate serialisability.

5.14.1 Validation conditions

Suppose transaction S is validated before transaction T .

No overlap If

$$\text{finish}(S) < \text{start}(T)$$

then:

- S completed before T began
- therefore, S cannot conflict with T

In this case, validation succeeds automatically.

Partial overlap If

$$\text{start}(T) < \text{finish}(S)$$

then S and T executed concurrently, so additional checks are required.

Two conditions must be met:

- no item written by S was read by T
 - this ensures that T did not read outdated values
- S completed its write phase before T began validation
 - this ensures that writes are applied serially and prevents write conflicts between S and T
 - If these conditions hold, then validating T preserves serialisability.

Otherwise:

- T is aborted
- its local workspace is discarded
- T may be restarted

5.14.2 Properties of optimistic validation

These validation conditions provide strong guarantees for serialisability.

However:

- they may reject some transactions unnecessarily
- therefore, transactions may restart more often than strictly required

In optimistic concurrency control, rollbacks are relatively inexpensive because:

- updates are stored only in local workspace copies until the write phase
- therefore, cascading rollbacks cannot occur

More precise validation tests are possible, but they require additional processing overhead.

Therefore:

- if conflicts are rare, optimistic methods are usually efficient
- if conflicts are frequent, conservative methods such as locking may perform better

5.15 Advanced transaction models

The concurrency control methods discussed so far are primarily designed for flat transactions.

A flat transaction:

- has a single start point and a single termination point
- either commits entirely or rolls back entirely

Flat transactions are well suited to traditional database systems, where:

- data items are relatively simple
 - e.g. integers, decimals, short strings
- transactions are short-lived
- failures during execution are relatively uncommon

However, modern database applications often involve:

- long-duration transactions
- large and complex data objects
- distributed or cooperative processing
- interactive user operations

These characteristics introduce several problems.

5.15.1 Problems with long transactions

Long transactions are more susceptible to failures because they execute over a longer period of time.

As a result:

- rolling back a transaction may discard a large amount of work
- transaction recovery becomes more expensive
- users may experience long delays

Ideally, recovery should restore the transaction to a state close to the point of failure, rather than restarting the entire transaction.

Long transactions may also access many data items.

To preserve isolation:

- large numbers of items may remain locked for extended periods
- this reduces concurrency and system throughput

Additionally:

- deadlocks become more likely
- transaction management becomes more complex

There are four main models that can be used to mitigate these problems:

- nested transaction model
- dynamic restructuring
- sagas
- workflow models

5.15.2 Nested transaction model

In the nested transaction model:

- a transaction can contain other transaction (sub-transactions)
- transactions form a tree (hierarchy) of sub-transactions
- each sub-transaction has its own start and termination point
- only leaf sub-transactions directly perform database operations

Transactions execute hierarchically:

- a parent transaction starts before its child transactions
- child transactions must complete before the parent transaction can complete

Thus, execution proceeds bottom-up with respect to commits.

Top-level transaction The top-level transaction is responsible for the overall outcome of the transaction tree.

For the root transaction:

- aborting causes all updates of the entire transaction tree to be undone
- committing makes all updates permanent
- the complete transaction must satisfy the ACID properties

Committed sub-transactions When a sub-transaction commits:

- its updates become visible only to its immediate parent
- its commit is conditional upon all ancestor transactions committing
- the sub-transaction is not durable on its own
 - updates become permanent only when the top-level transaction commits

Aborting sub-transactions If a sub-transaction aborts:

- only its parent transaction is informed
- higher-level transactions are unaffected unless the parent chooses otherwise

The parent may then - retry the sub-transaction - ignore the failure (if the sub-transaction is non-vital) - execute an alternative contingency sub-transaction - abort itself

Advantages There are many advantages of using the nested transaction model:

- modularity and finer-grained structure
 - transactions can be broken into smaller sub-units
 - easier to handle for concurrency/recovery
- increased concurrency
 - independent sub-transactions can execute in parallel
- improved fault isolation
 - failed sub-transactions can be rolled back without affecting others
 - reduces the impact of long-running transactions
- better recovery control
 - failures can be handled at a finer level than full transaction rollback

Savepoints Savepoints can be used to simulate nested transactions within a flat transaction model.

A savepoint is a named or identifiable point within a transaction representing a partially consistent

state.

It allows:

- partial rollback within a transaction
- restarting execution from an intermediate point instead of restarting the whole transaction

This provides:

- finer-grained recovery control than full transaction rollback
- improved efficiency in handling errors during long transactions

5.15.3 Sagas

A saga is an ordered sequence of flat transactions that may be interleaved with other transactions.

Each transaction T_i in the sequence has an associated compensating transaction C_i which semantically undoes the effects of T_i .

A saga consists of:

$$T_1, T_2, \dots, T_n$$

- If all transactions complete successfully:
 - the saga commits and all effects remain
- If transaction T_i fails:
 - previously completed transactions are compensated in reverse order:

$$T_1, T_2, \dots, T_n, C_i, C_{i-1}, \dots, C_1$$

This ensures the system returns to a consistent state, even though individual steps may have committed earlier.

Properties of sagas In contrast to flat transactions, sagas:

- relax isolation
 - intermediate results of each T_i may be visible to other transactions
- do not provide atomicity in the traditional sense
 - instead, they provide eventual consistency via compensation
- require compensating transactions
 - these are not true rollbacks, but logical undo operations

Sagas are useful when:

- sub-transactions are relatively independent
- compensating transactions can be produced easily

5.15.4 Dynamic restructuring

Dynamic restructuring is a transaction model that allows a running transaction T to be split into smaller serialisable sub-transactions.

- a transaction T can be split into two transactions A and B
- the split is only allowed if correctness (serialisability) is preserved
- the resulting transactions execute independently
- partial results may become visible to other transactions

This increases flexibility and concurrency in long-running transactions.

Split-transaction operation A split divides transaction T into:

$$A \rightarrow B$$

where A and B are serialisable sub-transactions.

To ensure correctness:

- A must appear before B in any equivalent serial schedule
- if A and B write the same data item, then all writes of A must occur before writes of B
- A cannot read values produced by B
- B may read values produced by A

This ensures A is serialised before B .

Failure handling If A aborts, then B must also abort. - however, if B aborts, A does not necessarily need to abort

This depends on whether B has become logically independent.

Independent split If there are no dependencies between A and B :

- no shared writes
- no shared read-write dependencies

Then:

- A and B may execute in any order (or even concurrently) without affecting correctness

Joint transaction operation The joint operation is the reverse of splitting:

- two transactions (or sub-transactions) are merged into a single transaction
- their operations are treated as part of one unified execution

Advantages of dynamic restructuring The advantages of dynamic restructuring are:

- adaptive recovery
 - parts of a transaction can commit independently
 - reduces the cost of full rollback
- reduced resource holding time
 - locks and resources can be released earlier
- better handling of long transactions
 - long operations are decomposed into manageable units
- increased concurrency
 - independent sub-transactions can execute in parallel

5.15.5 Workflow models

A workflow is a coordinated sequence of tasks carried out by people and/or computer systems to complete a complex business process.

A workflow model is a high-level transaction model designed for complex, long-running business processes.

Workflow systems generally do not strictly follow ACID properties.

6 Distributed databases

A distributed database is a collection of logically related data that is physically distributed across multiple sites connected by a network.

A distributed database management system (DDBMS) is a software system that manages a distributed database and makes the distribution transparent to users.

In a DDBMS:

- there is a single logical database composed of multiple fragments
- fragments may be stored at different sites across the network
- fragments can be replicated at multiple sites for reliability and availability
- each site typically runs its own local DBMS and has a degree of local autonomy
- users can access the global database as if it were a single database

A distributed database can be:

- partitioned (fragmented)
 - the database is split into disjoint fragments
 - each data item is stored at exactly one site
 - there is no replication and therefore no data redundancy
- completely replicated
 - a complete copy of the entire database is stored at every site
 - improves availability and reliability
 - can allow faster local data access and retrieval
- selectively replicated (partially replicated)
 - some fragments are replicated across multiple sites, while others are stored at a single site
 - combines aspects of partitioning and replication

6.1 Distributed transactions

A transaction in a DDBMS:

- is initiated at one site
- may execute across multiple sites
- is divided into sub-transactions, with each sub-transaction executed at a particular site

The DDBMS must ensure that:

- sub-transactions are correctly coordinated with local transactions at each site
- the ACID properties of both local and global transactions are preserved

All of the problems that appear in non-distributed databases (such as lost update, uncommitted dependency, inconsistent analysis) still exist, but additionally, there is the multiple-copy consistency problem:

All concurrency problems found in centralised databases (such as lost update, uncommitted dependency, and inconsistent analysis) can still occur in distributed databases. In addition, distributed databases introduce the multiple-copy consistency problem:

- if a replicated data item is updated, all copies of that item must be updated consistently
- otherwise, different sites may contain different values for the same data item, causing the global database to become inconsistent

6.2 Distributed serialisability

The concepts of schedules and serialisability naturally extend to distributed databases:

- a local schedule is the schedule of sub-transactions executed at a particular site
- the global schedule is the combined schedule of all transactions across all sites

A global schedule is serialisable if:

- each local schedule is serialisable at its site, and
- the serialisation order of transactions is consistent across all sites

In other words, if two transactions execute at multiple sites, they must appear in the same relative order in the equivalent serial schedule at every site where they both execute.

6.2.1 Testing serialisability

Given a distributed non-serial schedule:

- conflict serialisability can be tested using a precedence (serialisation) graph
- a schedule is conflict-serialisable if and only if its precedence graph is acyclic (has no directed cycles)
- this is the same criterion used for centralised (single-site) schedules

However, in a distributed database this introduces a practical challenge:

- the database is spread across multiple sites
- no single site has a complete view of all transactions and their conflicts
- therefore, constructing the full global precedence graph is non-trivial and requires coordination between sites

6.2.2 Ensuring serialisability

If all transactions are local and there is no data replication:

- standard centralised concurrency control protocols can be used, such as:
 - two-phase locking (2PL)
 - timestamp ordering

If there are distributed transactions and/or replicated data, these protocols must be extended to maintain global serialisability.

There are several distributed variants of two-phase locking (2PL):

- centralised 2PL
- primary copy 2PL
- distributed 2PL
- majority 2PL

6.2.3 Centralised 2PL

In centralised two-phase locking:

- a single site acts as a central lock manager (LM)
- the LM maintains all lock information for the entire distributed database

When a transaction is initiated at a site S_i :

- the local transaction coordinator (TC) at S_i manages the transaction's execution
- if the transaction needs to access a data item x :
 - the TC requests the appropriate lock (shared or exclusive) from the central LM
 - the LM grants or denies the request according to standard 2PL rules

The main idea of centralised 2PL is to treat the database if it were a single centralised database.

Advantages

- simple implementation
 - essentially reuses centralised 2PL logic without major distributed –coordination
- simplified deadlock handling
 - the LM can construct a global wait-for graph and detect deadlocks centrally

Disadvantages

- bottleneck
 - the central LM can become a performance and scalability limitation
- low availability
 - if the central LM fails, lock coordination stops and the entire system may become unavailable

6.2.4 Primary copy 2PL

Primary copy 2PL is an extension of centralised locking for replicated databases:

- the database is partitioned into groups of data items
- each group (or primary copy set) has a designated primary copy site
- each primary copy site acts as a lock manager for its data items
 - there are multiple LMs

For every replicated data item x :

- one copy is designated as the primary copy
- all other copies are secondary (slave) copies

When a transaction needs to access a data item x :

- the transaction coordinator (TC) identifies the primary copy site of x
- lock requests (read or write) are sent to the LM of the primary site
- the primary site enforces 2PL for all operations on x

For updates:

- the primary copy is updated first under an exclusive lock

- updates are then propagated to secondary copies (immediately or lazily, depending on the system)

The key idea is that locking responsibility is distributed across sites, but centralised per data item (via its primary copy site).

Advantages

- better scalability than centralised 2PL
 - lock management is distributed across multiple primary sites
- reduced bottleneck
 - no single global lock manager
- good performance when write locality exists
 - updates are handled at the relevant primary site

Disadvantages

- still partially centralised
 - each primary copy site is a local bottleneck for its data items
- failure sensitivity
 - failure of a primary copy site makes its data items temporarily inaccessible for updates (and possibly reads)
- replication overhead
 - updates must still be propagated to secondary copies

Backup sites To improve reliability, a backup site can be assigned to each primary copy site:

- each primary site (and its lock manager) has a designated backup site
- the backup maintains a log of lock and update activity (or receives updates continuously)

If a primary site fails:

- the backup detects the failure (e.g., via missing communications from the primary site)
- the backup takes over as the new primary lock manager
- transactions are redirected to the new primary site
- the new primary continues processing and informs other sites of the change

When the failed site recovers:

- it synchronises with the current primary (or backup, depending on system state)
- it updates its state using the logged operations
- it may resume as primary again after coordination

6.2.5 Distributed 2PL

In the distributed 2PL protocol:

- each site has its own local lock manager (LM)
- each LM is responsible for the data items stored at that site

If data is not replicated:

- distributed 2PL behaves like standard local 2PL at each site
- there is no cross-site locking coordination required

If there is data replication:

- the read-one-write-all (ROWA) rule is used:
 - read operations access one available replica
 - * a shared lock is taken at a single site holding a copy
 - write operations must access all replicas
 - * exclusive locks are required at every site holding a copy of the item

For a write operation to proceed:

- the transaction must obtain confirmation (lock grants) from all relevant sites
- if any site cannot grant the lock, the write must wait or abort (depending on policy)

The key difference between distributed 2PL and primary copy 2PL is:

- distributed 2PL: all replicas are kept strictly consistent (synchronous updates)
- primary copy 2PL: only the primary site enforces locking; updates are then propagated to other replicas
- distributed 2PL typically incurs higher communication overhead due to multi-site locking

6.2.6 Majority 2PL

Majority 2PL is an extension of distributed 2PL:

- each data item is replicated across multiple sites
- to write, a transaction must acquire locks from a majority of replicas (rather than all)

A read-lock can be simultaneously held by many users.

A write-lock can be held by only one user at a time.

6.3 Distributed deadlocks

In a distributed database:

- each site maintains a local wait-for graph
- these local graphs may not show any cycle (so no local deadlock is detected)
- however, when combined, the global wait-for graph may contain a cycle, indicating a distributed (global) deadlock

This happens because dependencies between transactions can span multiple sites, so no single site has complete information.

Therefore, specialised schemes are required to detect deadlocks that cannot be identified locally.

Three main approaches are:

- centralised deadlock detection
- hierarchical deadlock detection
- distributed deadlock detection

6.3.1 Centralised deadlock detection

In centralised deadlock detection:

- a single site acts as the Deadlock Detection Coordinator (DDC)

- the DDC maintains a global wait-for graph (WFG)

Periodically:

- each site sends its local wait-for graph (or changes to it) to the DDC
 - to reduce communication overhead, only updates may be transmitted instead of the full graph
- the DDC merges all received information to construct or update the global WFG
- the DDC checks the global WFG for cycles

If a cycle (deadlock) is detected:

- the DDC selects a victim transaction
- the victim is aborted or rolled back to break the deadlock

The main drawback is that the DDC is a single point of failure:

- if it fails, deadlock detection stops for the entire system

6.3.2 Hierarchical deadlock detection

In hierarchical deadlock detection:

- sites are organised into a logical hierarchy (tree structure)
- each parent node is responsible for detecting deadlocks across its child nodes
- each site sends its local wait-for graph (WFG) to its parent in the hierarchy

At different levels:

- Level 1 (leaf level):
 - individual sites perform local deadlock detection
 - each site builds its local WFG
- Intermediate levels:
 - each parent node receives WFGs from its child nodes
 - it merges them into a combined WFG for that subtree
 - it detects deadlocks involving transactions across those child sites
- Root level:
 - receives aggregated WFGs from all subtrees
 - detects global deadlocks involving all sites

There is no dependency on a centralised detection site, so:

- it is more reliable
- there is less communication costs

However, it is more complex to implement.

6.3.3 Distributed deadlock detection

In distributed deadlock detection, every site S maintains:

- a local wait-for graph (WFG) for transactions at site S
- an external node T_{ext} , representing all transactions outside site S

This is used to capture dependencies involving remote sites.

For a transaction T_i in the local WFG at site S :

- if T_i is waiting for a lock held by a transaction at another site S' :
 - add an edge $T_i \rightarrow T_{ext}$ labelled with S'
- if a transaction at another site S' is waiting for a lock held by T_i :
 - add an edge $T_{ext} \rightarrow T_i$ labelled with S'

At each site S :

- if there is a cycle that does not involve T_{ext} :
 - there is a local deadlock
- if there is a cycle involving T_{ext} :
 - this indicates a potential global deadlock involving remote sites
 - however, it may be a false positive

To confirm whether a global deadlock exists:

- sites exchange information about cycles involving T_{ext}
- using the edge labels, the system expands and merges partial cycles across sites
- this continues until either:
 - a global cycle without T_{ext} is found, confirming deadlock, or
 - the cycle cannot be extended, so no global deadlock exists

Distributed deadlock detection is more robust than centralised approaches

- it has no single point of failure

However, it has a high communication overhead:

- many messages are required to resolve potential cycles

6.4 Network partitioning

A critical failure that can occur in distributed database systems is network partitioning.

Network partitioning occurs when communication failures split the network into two or more isolated partitions that cannot communicate with each other.

In distributed systems, it may be difficult to determine whether:

- a site has failed, or
- the communication network between sites has failed

For example, suppose node S_1 cannot reach not S_2 within a fixed (timeout) period. Then, the cause could be one of many:

- S_2 has crashed (failed)
- the communication link $S_1 - S_2$ has failed
- the network has become partitioned
- S_2 is overloaded or slow, causing delayed responses

Consider a distributed transaction T executing across multiple sites in a partitioned network.

If:

- the sub-transactions of T in one partition commit, while

- the sub-transactions in another partition abort,

then the atomicity property of transaction T is violated.

A distributed transaction must either:

- commit at all participating sites, or
- abort at all participating sites

Therefore, distributed commit protocols are required to ensure that:

- all participating sites agree on whether to commit or abort
- only then is the global transaction finalised

In a distributed transaction:

- the site where the transaction is initiated acts as the transaction coordinator (TC)
- all other involved sites are called participants
- the TC knows all participants
- each participant knows the TC

6.4.1 Two-phase commit

The two-phase commit (2PC) protocol is used in distributed databases to ensure that all participants in a distributed transaction either:

- commit together, or
- abort together

The protocol works as follows:

- the TC asks all participants if they are ready commit
- if one participant votes to abort, or provides no timely response:
 - the TC tells all participants to abort
- if all participants vote to commit:
 - the TC tells all participants to commit

All participants must follow the final global decision made by the TC.

Coordinator procedure The coordinator executes the protocol in two phases:

- Voting
 - the TC writes a **begin_commit** record to its log file
 - it sends a **PREPARE** message to all participants, and
 - waits for responses within a timeout period
- Decision
 - if any participant votes **ABORT**, or fails to respond in time, the TC:
 - * writes an **abort** record to its log
 - * send **GLOBAL_ABORT** to all participants
 - * wait for acknowledgements (acks)
 - if all participants vote **READY_COMMIT**, the TC:
 - * writes a **commit** record to its log
 - * send **GLOBAL_COMMIT** to all participants
 - * wait for acks

- once all acks are received:
 - * the TC writes an `end_transaction` record to its log file
 - * transaction resources can then be released
- if a participant does not acknowledge:
 - * the TC repeatedly resends the global decision until the acknowledgement is received

Participant procedure The procedure executed by each participant in the 2PC protocol is:

- when the participant it receives a `PREPARE` message:
 - it decides whether it can commit its part of the transaction:
 - * if it can, write a `ready_commit` record to the log & send `READY_COMMIT` to the TC,
 - * if it can't, write an `abort` record to the log & send `ABORT` to the TC
- if a `GLOBAL_ABORT` message is recieved, the participant:
 - writes an `abort` record to its log
 - abort the sub-transaction
 - sends an ack to the coordinator
- if a `GLOBAL_COMMIT` message is recieved, the participant:
 - writes a `commit` record to its log
 - commit the sub-transaction
 - send an ack to the coordinator

2PC termination protocols A 2PC termination protocol is used when a coordinator or participant fails to receive an expected message within a timeout period.

Its purpose is to ensure that the distributed transaction eventually reaches a consistent outcome despite failures or communication problems.

Coordinator For the coordinator, a time out can only happen in the `WAITING` or `DECIDED` stages.

If a timeout happens in the `WAITING` stage:

- the TC is waiting for participants to vote to commit or abort
- it cannot commit without receiving all votes, so it decides to abort

If a timeout happens in the `DECIDED` stage:

- the TC is waiting for participants to send an ack confirming they followed the global decision
- it resends the global decision to any sites that didn't send an ack until they do

Participant The simplest participant strategy is:

- remain blocked until communication with the coordinator is restored

However, this can lead to long blocking delays.

More efficient timeout handling depends on the participant's current state.

If a timeout happens in the `INITIAL` stage:

- the participant is waiting for a `PREPARE` message
- the participant can safely abort its sub-transaction
 - no commit decision could have been made involving it

If a timeout happens in the `PREPARED` stage:

- the participant has already voted to commit
- it is waiting for the global decision
- the participant cannot independently decide to commit or abort
 - doing so could violate atomicity
- without further information, the participant is blocked
 - it must wait for the coordinator to recover, or
 - attempt to communicate with other participants to discover the global decision

2PC recovery protocols A 2PC recovery protocol is invoked when a coordinator or participant restarts after a failure.

Recovery is based on the contents of the site's log file, which records the state of the transaction before failure.

Coordinator If the coordinator failed during **INITIAL**:

- the coordinator had not yet started the commit protocol
- recovery just starts the commit procedure

If the coordinator failed during **WAITING**:

- it has sent **PREPARE** but not yet received all votes
- it restarts the commit procedure

If the coordinator failed during **DECIDED**:

- it has sent the global decision
- if all acks have been received, the transaction is completed successfully
- otherwise, it resends the global decisions and waits for acks

Participants If the participant failed during **INITIAL**:

- it has not yet voted, so a global decision could not have been made
- it can unilaterally abort its sub-transaction

If the participant failed during **PREPARED**:

- the participant has already voted to commit and is waiting for the global decision
- it initiates the termination protocol
 - it attempts to contact the coordinator or other participants to discover the final global decision
 - until the decision is known, the participant remains blocked

If the participant failed during **ABORTED** or **COMMITTED**:

- the participant had already completed its part of the transaction
- no action is needed